

Predicting Drug Solubility from Molecular Descriptors

Arun Pravin A P · B Harshith Varma · Monish U

Amrita School of Computing
Amrita Vishwa Vidyapeetham, Coimbatore

Agenda

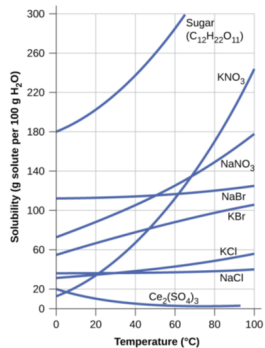
1. Introduction & Motivation
2. The AqSolDB Dataset
3. Exploratory Data Analysis
4. Preprocessing & Feature Engineering
5. Regression Algorithms
6. Results & Conclusion

Goal: Build a machine learning regression model that can predict a drug's **water solubility (LogS)** using its chemical and molecular features.

Understanding Chemical Solubility

A drug's ability to dissolve in water depends heavily on its chemical properties:

- **Molecular Weight/Volume:** Bigger and heavier molecules generally find it harder to dissolve.
- **Lipophilicity (MolLogP):** Tells us how much the drug prefers oil over water.
- **Polarity (TPSA):** The ability of the molecule to form hydrogen bonds with water.
- **Structural Complexity:** Features like the number of rings and rotatable bonds affect how the molecule moves and interacts.

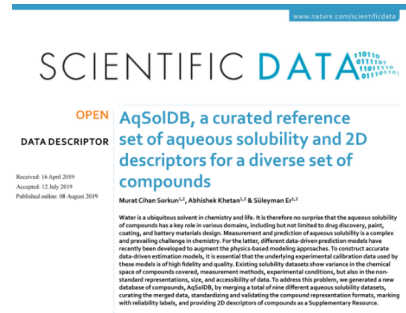


AqSolDB Dataset

AqSolDB is a well-known, open-access database containing the aqueous solubility values for **9,982 unique chemical compounds**.

It was created by researchers from the Autonomous Energy Materials Discovery group and published in the *Nature Scientific Data* journal in 2019.

It is widely used as a standard benchmark dataset for applying machine learning in drug discovery and chemistry.



AqSolDB – Columns (1/3)

Column	Type	Description
ID	String	Unique identifier for each compound.
Name	String	Common or scientific name of the chemical.
InChI	String	A standard text representation of the molecule's structure.
InChIKey	String	A short, hashed version of the InChI string used for easy searching.
SMILES	String	A text-based format that defines atoms, bonds, and rings.
Solubility	Float	Log-scaled aqueous solubility (LogS). This is the value we want to predict.
SD	Float	Standard deviation of the solubility measurement across different lab experiments.
Ocurrences	Integer	Number of times the compound appeared in the original source datasets.
Group	String	A category label assigned by the original researchers for cross-validation.

AqSolDB – Columns (2/3)

Column	Type	Description
MolWt	Float	Molecular weight (g/mol).
MolLogP	Float	Partition coefficient (logP); tells us how hydrophobic (water-fearing) it is.
MolMR	Float	Molar refractivity; related to the volume and size of the molecule.
HeavyAtomCount	Float	Total number of non-hydrogen atoms.
NumHAcceptors	Float	Number of atoms (like Oxygen or Nitrogen) that can accept hydrogen bonds.
NumHDonors	Float	Number of groups (like O-H or N-H) that can donate hydrogen bonds.
NumHeteroatoms	Float	Number of atoms that are not carbon or hydrogen.
NumRotatableBonds	Float	Number of single bonds that allow the molecule to rotate freely.
NumValenceElectrons	Float	Total valence electrons involved in the molecule's bonding.

AqSolDB – Columns (3/3)

Column	Type	Description
NumAromaticRings	Float	Number of flat, ring-like structures (e.g., benzene).
NumSaturatedRings	Float	Number of rings that only have single bonds.
NumAliphaticRings	Float	Number of non-aromatic rings.
RingCount	Float	Total number of rings in the molecule.
TPSA	Float	Topological Polar Surface Area; measures the polar area of the molecule.
LabuteASA	Float	An estimate of the molecule's surface area.
BalabanJ	Float	A mathematical score showing how highly branched the molecule is.
BertzCT	Float	A score that measures the overall structural complexity of the molecule.

Part A – Exploratory Data Analysis

Understanding the Data and Checking Quality

Missing Values

Since this is a highly standard dataset published in *Nature*, it is extremely clean. We checked the entire dataset, and there are absolutely no missing values.

Total Missing Values

0

(Checked across 9,982 rows $\times$ 25 columns)

Target Variable & Dataset Check

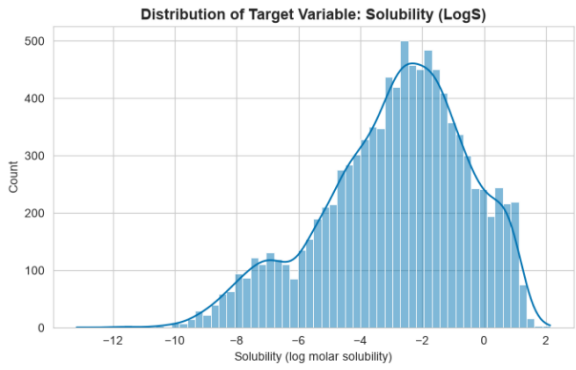
Solubility (LogS) – Summary Statistics:

count	mean	std	min	25%	50%	75%	max
9,982	-2.890	2.368	-13.172	-4.326	-2.618	-1.210	2.138

Exact Duplicate Rows: 0 | Duplicate Compounds (by InChIKey): 0

Since there are no missing values and no duplicates, we do not need to do any imputation. The target variable Solubility ranges from -13.2 (very insoluble) to $+2.1$ (very soluble).

Distribution of the Target Variable



The target looks somewhat like a bell curve, but it has a long tail on the left side (negative skew).

This happens because a small number of drugs in the dataset are extremely insoluble. While simple linear models might struggle slightly with this skew, tree-based models like Random Forest can handle it very easily.

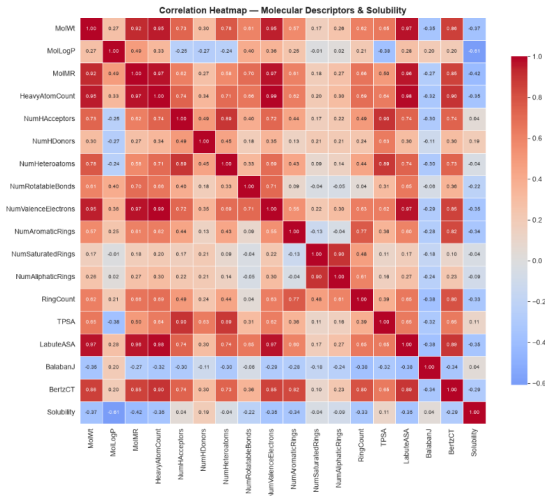


Understanding the Feature Distributions

- Most continuous features like MolWt and TPSA are **right-skewed**. This simply means most drugs in our dataset are small to medium-sized, with only a few very heavy molecules.
- Features like the number of rings or hydrogen donors have small integer values, which is normal for standard drug molecules.
- Complexity scores like BertzCT and BalabanJ show a wide variety of values.

Overall, the dataset has a very good variety of chemicals. This rich diversity gives our machine learning models plenty of information to learn from.

Correlation Heatmap

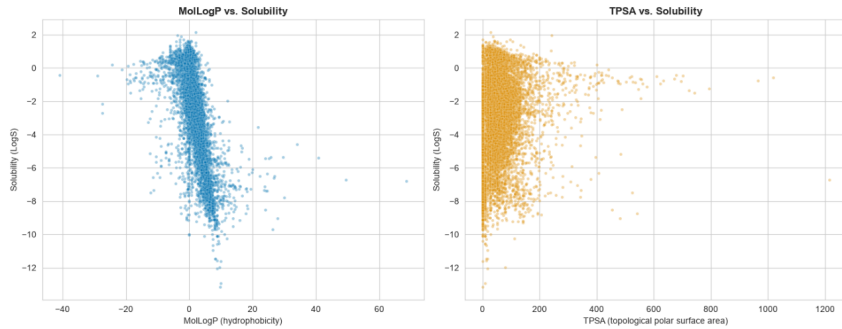


Understanding the Correlation Heatmap

- **Top Predictor:** MolLogP (lipophilicity) has the strongest negative correlation with Solubility ($r = -0.61$). This makes perfect sense: oily drugs do not dissolve well in water.
- **Other Predictors:** Features related to size, like MolMR and MolWt, also show moderate negative correlations.
- **Multicollinearity:** Some features are basically giving the same information. For example, Molecular Weight and Heavy Atom Count are 97% correlated.

Because of this high correlation between features (multicollinearity), simple linear models might get confused. We will need to use Regularization techniques (like Ridge or Lasso) to keep them stable.

Feature vs. Target Scatter Plots



MolLogP & TPSA vs. Solubility

MolLogP vs. Solubility

- Shows a clear **negative trend**. As the drug becomes more oil-loving, its solubility drops.
- However, the points are widely spread out. This means MolLogP alone is not enough to get a perfect prediction.

TPSA vs. Solubility

- Shows a very weak **positive trend** ($r = +0.11$).
- Molecules with the exact same TPSA value have vastly different solubilities, meaning this feature needs help from others.

Takeaway: No single feature is enough to predict solubility perfectly. Our machine learning models will need to intelligently combine multiple features to get accurate results.

Part B – Preprocessing and Feature Engineering

Cleaning Data and Creating New Features

Data Cleaning Steps

How we cleaned the data:

1. **Checking for Duplicates:** We confirmed there are no duplicate compounds in the dataset.
2. **Removing Confusing Data:** We dropped 155 rows where the solubility measurements had a very high standard deviation ($SD > 1.0$). Teaching the model with unstable, confusing data will only lower its accuracy.
3. **Handling Outliers:** A few molecules had extremely high values for MolLogP and MolMR. We capped these extreme values at the 1st and 99th percentiles (a process called Winsorization) so they don't drag the model's performance down.

Rows removed due to confusing data: **155** (1.55%) | Total rows ready for modelling: **9,827**

Dropping Columns and Splitting Data

- **Dropping Text Columns:** We removed columns like ID, Name, and SMILES. These are just text labels and do not provide numerical value for the algorithms.
- **Preventing Data Leakage:** We dropped metadata columns like SD, Occurrences, and Group to ensure the model doesn't cheat by looking at experimental details.
- **Train-Test Split:** We split the data into **80% for Training** and **20% for Testing**.
- **Stratified Split:** To ensure both sets have a fair mix of high and low solubility drugs, we divided the target into 10 bins and split the data evenly across them.

Train–Test Split & Safe Winsorization

17 Predictive Features → Continuous Target (Solubility)

Training Data: 7,861 rows

Testing Data: 1,966 rows

Train Target Mean: -2.872

Test Target Mean: -2.861

Capping Limits (From Train Data Only):

MolLogP: $[-9.05, 10.89]$ → capped 158 train, 42 test

MolMR: $[5.75, 229.69]$ → capped 155 train, 52 test

Very Important: We calculated the capping limits (1st and 99th percentiles) only on the Training data, and then applied those exact same limits to the Test data. This ensures the Test data remains completely unseen.

Feature Engineering – TPSA_per_MolWt

TPSA tells us the total polar surface area of a molecule. But let us look at an example:

- Molecule A: $\text{TPSA} = 50$, $\text{MolWt} = 100 \rightarrow$ **High Polarity Density** (0.50)
- Molecule B: $\text{TPSA} = 50$, $\text{MolWt} = 500 \rightarrow$ **Low Polarity Density** (0.10)

Both have the same TPSA, but Molecule A is much smaller, meaning it is more polar for its size.

New Feature: We created `TPSA_per_MolWt` by dividing TPSA by Molecular Weight. This new feature has a much better correlation with solubility (**0.467**) compared to just TPSA alone (0.109).

Standard Scaling

Many algorithms (like SVR and KNN) calculate the mathematical distance between data points.

If a feature like `MolWt` has values in the hundreds, and `NumHDonors` has values from 0 to 10, the larger numbers will completely dominate the calculations.

To fix this, we used `StandardScaler` to bring all features to a common scale (mean of 0, standard deviation of 1).

Again, we fit the scaler **only on the training data** to avoid any data leakage, and then transformed both the train and test sets.

Final Preprocessed Dataset

We now have 18 clean, scaled numerical features ready to predict the Solubility target.

	MolWt	LogP	MolMR	HAC	HAcc	HDon	HetA	RotB	VE	ArR	SatR	AliR	Ring	TPSA	LASA	BalJ	BertzCT
5785	0.70	1.54	1.26	0.88	0.15	-0.74	-0.25	1.80	1.00	-0.05	-0.33	-0.43	-0.31	-0.16	0.82	-0.01	0.11
4868	-0.03	0.04	-0.34	-0.27	-0.14	-0.07	-0.04	-0.36	-0.31	-0.05	-0.33	-0.43	-0.31	0.02	-0.26	0.69	-0.14
6932	0.72	1.20	0.44	0.06	0.15	-0.74	0.60	0.36	0.13	-0.05	-0.33	-0.43	-0.31	-0.55	0.29	0.56	-0.00

This fully processed data is what we will feed into all our machine learning algorithms in the next section.

Regression Algorithms

Predictive Modelling: Ten Algorithms Assessed

Linear Regression
& Regularization

Polynomial
Regression

Decision Tree
Regressor

Random Forest
&
Gradient
Boosting

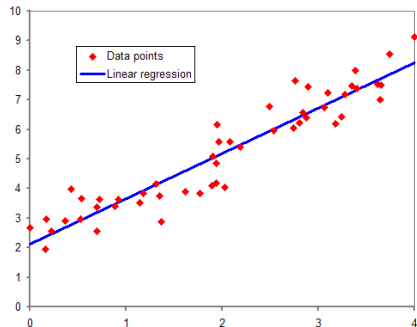
Support Vector
& KNN

Linear Regression

How it works: Linear Regression tries to draw the best straight line (or flat plane) through our data points to predict solubility.

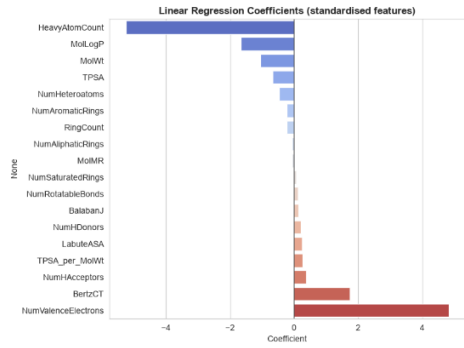
It gives a mathematical weight to each of our 18 features, trying to minimize the errors between its predictions and the actual values:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$



Drawback: It assumes the relationship between a feature and solubility is perfectly straight. But in real chemistry, properties often interact in complex, curved ways.

Linear Regression – Results



$$R^2 = 0.5566 \quad \text{RMSE} = 1.5664 \quad \text{MAE} = 1.1459$$

This is our baseline (starting) model. It explains about 56% of the variance in the unseen test set.

The model learned correctly: heavier molecules (HeavyAtomCount) and oilier molecules (MolLogP) get negative weights, meaning they lower the solubility.

The Need for Regularization

The Problem: In our EDA, we saw that some features (like Molecular Weight and Heavy Atom Count) are highly correlated. This confuses simple Linear Regression, making it assign unstable, huge weights to balance the math.

The Solution (Regularization): We add a penalty to the model to keep it disciplined. It forces the model to keep the weights small and stable.

- **Ridge (L2):** Shrinks all weights smoothly, preventing unstable values without dropping any features.
- **Lasso (L1):** Can shrink the weights of useless features to exactly zero. This means it automatically performs Feature Selection.
- **ElasticNet:** Mixes both Ridge and Lasso methods together.

Ridge Regression

How it works: Ridge adds an L2 penalty to the loss function.

$$\text{Loss} = \text{RSS} + \alpha \sum_{j=1}^p \beta_j^2$$

By penalizing the square of the weights, it forces the model to shrink the weights of highly correlated features evenly. This makes the model much more stable.

Optimal $\alpha = 0.1$
(Selected via 5-Fold CV)

Metric	Ridge Regression
Test Set R^2	0.5567
RMSE	1.5661
MAE	1.1459

Result: Ridge effectively stabilized the correlated features while keeping all 18 features active, giving a very tiny improvement over the basic Linear Regression.

Lasso Regression

How it works: Lasso uses an L1 penalty instead.

$$\text{Loss} = \text{RSS} + \alpha \sum_{j=1}^p |\beta_j|$$

The special thing about Lasso is that it can shrink the weights of unimportant features to exactly zero, effectively dropping them from the model.

Optimal $\alpha = 0.001$
(Selected via 5-Fold CV)

Metric	Lasso Regression
Test Set R^2	0.5588
RMSE	1.5625
MAE	1.1461

Result: Lasso successfully performed feature selection. It completely dropped NumAliphaticRings and LabuteASA (setting their weights to 0), proving they were not needed to get a good prediction.

ElasticNet Regression

How it works: ElasticNet combines both Ridge and Lasso penalties.

It uses an `l1_ratio` parameter. A ratio of 1.0 means it acts exactly like Lasso, while 0.0 means it acts exactly like Ridge. We tuned this to find the perfect balance.

Optimal $\alpha = 0.001$, `l1_ratio` = 0.9
(Selected via 5-Fold CV)

Metric	ElasticNet
Test Set R^2	0.5589
RMSE	1.5622
MAE	1.1462

Result: The tuning process heavily favoured the Lasso penalty (90% L1).

Takeaway on Regularization: All regularized models stopped around 56% accuracy. This tells us the problem is not just multicollinearity; a straight line simply cannot fit this complex chemical data well.

Polynomial Regression: Capturing Curves

How it works: To capture curves, Polynomial Regression creates new features by squaring the existing ones (x^2) and multiplying them together ($x_1 \times x_2$).

This allows the straight line to bend. For example, it helps the model learn if the effect of lipophilicity changes depending on how heavy the molecule is.

Degree	Feature Count	CV R^2
1	4	0.5119
2	14	0.6311
3	34	-1.2959 (Overfit)

Test Results (Degree 2)
 $R^2 = 0.6214$ RMSE = 1.4474

Result: Accuracy jumped from 55% to 62%! This clearly proves that chemical features interact with each other to decide solubility, and we need models that handle non-linear curves.

Decision Tree Regressor

How it works: A Decision Tree works like a flowchart. It splits the data by asking simple Yes/No questions based on feature values (for example, 'Is MolLogP < 2.0?').

- It checks all features to find the split that reduces the error the most.
- It keeps splitting until it reaches the end (a leaf).
- For a new drug, it follows the flowchart down to a leaf and predicts the average solubility of the training samples in that leaf.

Controlling Overfitting: If we let the tree grow too deep, it will just memorize the training data perfectly but fail on new data. We control this using the `max_depth` parameter.

Decision Tree – Tuning and Results



As the depth increases, the **Training** accuracy keeps going up to 100%. However, the **Validation** accuracy peaks at **depth 7** and then drops. This is a classic sign of overfitting.

Decision Tree (depth 7)

$$R^2 = 0.6798$$

$$\text{RMSE} = 1.3311$$

$$\text{MAE} = 0.9335$$

Result: At depth 7, we get a solid 67.9% accuracy. Trees naturally understand thresholds and feature interactions better than polynomial math.

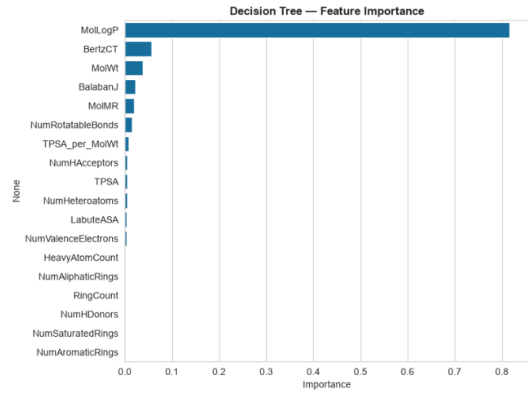
Decision Tree – Feature Importance

Decision Tree Regressor

$$R^2 = 0.6798 \mid \text{RMSE} = 1.3311 \mid \text{MAE} = 0.9335$$

Insight: Cross-validation identifies `max_depth = 7` as the optimal threshold (peak CV $R^2 = 0.6858$), beyond which the tree begins overfitting the training data (Train R^2 continues rising while CV R^2 drops).

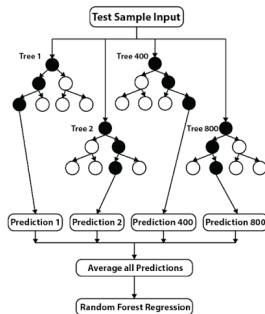
MolLogP is the most important feature. The tree uses it the most to make its splits, confirming it as the primary descriptor.



Random Forest

How it works: A single Decision Tree is good, but it can be unstable and overfit easily. Random Forest solves this by creating a "forest" of many trees.

- We build 300 different Decision Trees.
- Each tree gets a random sample of the data and a random subset of features to look at.
- For a final prediction, we simply take the **average** answer from all 300 trees.



Advantage: Because we are averaging the answers of many different trees, the overall model becomes very stable, accurate, and resistant to overfitting.

Random Forest – Tuning & Results

We used GridSearchCV with **3-fold CV** to test different numbers of trees (`n_estimators`) and tree depths (`max_depth`).

Best Parameters: 300 trees, `max_depth` = 25

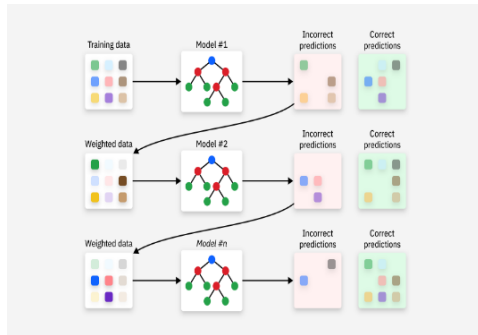
Metric	Random Forest Score
Test Set R^2	0.7606
RMSE	1.1508
MAE	0.7710

Result: Random Forest gave us a massive jump in accuracy. It captures **76%** of the variance on the test set, proving that averaging multiple trees works wonderfully for this complex dataset.

Gradient Boosting

How it works: Instead of building 300 independent trees at the same time, Gradient Boosting builds trees **one by one** to fix mistakes.

- The first tree makes a guess.
- We calculate the errors (mistakes) of that first tree.
- The second tree is trained specifically to predict and fix the first tree's mistakes.
- The third tree fixes the second tree's mistakes, and so on.



Gradient Boosting – Tuning & Results

We tuned the `learning_rate` (how fast it learns) and `n_estimators` (number of trees) using 3-fold cross-validation.

Best Parameters: `learning_rate = 0.1`, `n_estimators = 200`

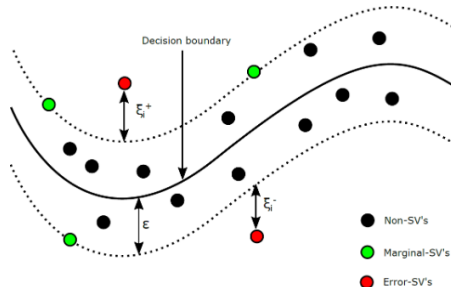
Metric	Gradient Boosting Score
Test Set R^2	0.7444
RMSE	1.1892
MAE	0.8354

Result: It achieved a very strong 74.4% accuracy. It is slightly behind Random Forest, possibly because we only tuned it up to 200 trees to save time.

Support Vector Regression (SVR)

How it works: SVR creates a "tube of tolerance" around the predictions.

- If a prediction falls inside this tube, SVR doesn't count it as an error.
- It only penalizes the model for points that fall completely outside the tube (the support vectors).
- By using the **RBF Kernel**, SVR can easily bend this tube to fit highly curved, non-linear data.



SVR – Tuning & Results

We optimized the error penalty (C) and the kernel type (Linear vs RBF).

Best Parameters: $C = 10$, kernel = RBF

Metric	SVR (RBF Kernel)
Test Set R^2	0.7540
RMSE	1.1668
MAE	0.7914

Result: SVR with the RBF kernel performed brilliantly (75.4%), coming in as our second-best model. This confirms again that the data has strong non-linear patterns.

K-Nearest Neighbors (KNN) Regressor

How it works: KNN is a very simple distance-based model.

To predict the solubility of a new drug:

1. It calculates the distance between the new drug and all drugs in the training set.
2. It finds the K most similar drugs (nearest neighbours).
3. It simply takes the average of their actual solubility values.

Why scaling is critical: If we don't scale the data, a feature with big numbers (like Molecular Weight) will completely dominate the distance calculation over a feature with small numbers (like Ring Count).

KNN Regressor – Tuning & Results

We tested different values for K (number of neighbours) using 5-fold cross-validation.

K Parameter	3	5	7	9	11	15
Mean CV R^2	0.7148	0.7302	0.7285	0.7293	0.7274	0.7215

Evaluation Without Scaling: $R^2 = 0.5787$ | With Standard Scaling ($K=5$): $R^2 = 0.7264$

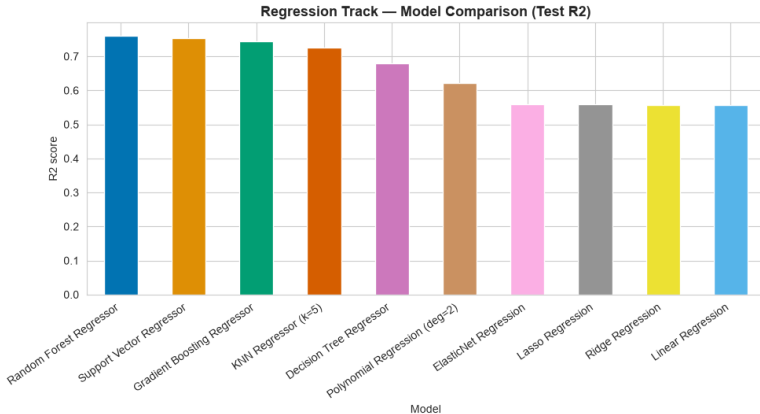
Metric	KNN Regressor ($K=5$)
Test Set R^2	0.7264
RMSE	1.2304

Result: As expected, without scaling, KNN got only 57.8%. After applying Standard Scaler, it jumped to 72.6%. Scaling is extremely important for distance-based models!

Comparing All 10 Algorithms

Model Framework	Test R^2	RMSE	MAE
Random Forest Regressor	0.7606	1.1508	0.7710
Support Vector Regressor (RBF)	0.7540	1.1668	0.7914
Gradient Boosting Regressor	0.7444	1.1892	0.8354
KNN Regressor (k=5)	0.7264	1.2304	0.8411
Decision Tree Regressor	0.6798	1.3311	0.9335
Polynomial Regression (deg=2)	0.6214	1.4474	1.0565
ElasticNet Regression	0.5589	1.5622	1.1462
Lasso Regression	0.5588	1.5625	1.1461
Ridge Regression	0.5567	1.5661	1.1459
Linear Regression (OLS)	0.5566	1.5664	1.1459

Model Comparison – Test R^2



5-Fold CV Verification – Top 2 Models

Based on the test results, **Random Forest** and **SVR** are our best models. We ran a 5-fold cross-validation to make sure they are truly stable and not just getting lucky on the test set.

Random Forest:

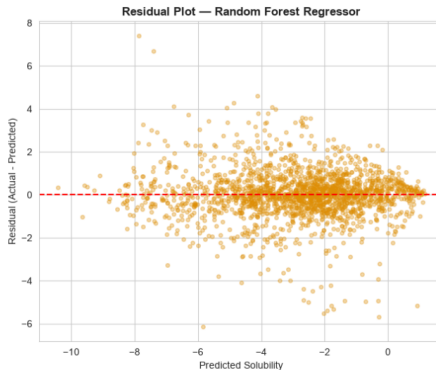
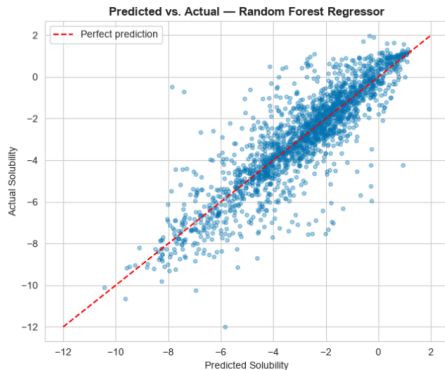
Scores = [0.776, 0.763, 0.795, 0.772, 0.760] Mean R^2 = 0.7731, Std = 0.0122

SVR (RBF):

Scores = [0.762, 0.745, 0.766, 0.768, 0.754] Mean R^2 = 0.7590, Std = 0.0087

Result: Both models proved to be very consistent across all folds, with Random Forest taking the clear lead.

Best Model Diagnostics – Random Forest



Final Conclusion

- **Simple lines don't work:** Linear Regression and its regularized versions (Ridge/Lasso) stopped around 56% accuracy. Chemical properties interact with each other, meaning a straight line cannot predict solubility accurately.
- **Curves improve accuracy:** Adding polynomial features jumped the score to 62%, proving that we need models that can handle curved relationships.
- **Random Forest is the clear winner:** By averaging 300 decision trees, Random Forest naturally captured the complex interactions between features. It gave us the highest accuracy (76.06%) and was very stable during cross-validation.
- **Struggles with extreme values:** The diagnostic graph shows the model's predictions get slightly scattered for extremely insoluble drugs. This is likely because we have very few examples of these extreme drugs in the dataset to learn from.

Thank You